Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Kayıt iki parçadan oluşur ve parçalar arşivde ters sırada birleşmiştir. Dersin tanıtım bölümü kaydın ikinci yarısındadır (01:50:41 sonrası). Teknik ders kaydın başındadır (00:00:33 - 01:48:30). Yani tanıtım önce, teknik ders sonra yapılmıştır.

Genel Konular

- Veri Yapıları ve Algoritmalar dersinin amacı ve kapsamı (01:51:18 sonrası).
- Veri yapılarının gerçek hayattaki kullanımları: graf (graph), yığın (stack), kuyruk (queue).
- Algoritma tasarım adımları: problemi anla, örnek çöz, tasarla, doğrula, analiz et, sonra kodla.
- İyi bir algoritmanın ölçütleri: verimlilik (zaman ve yer), basitlik (simplicity), genellik (generality).
- Sırasız dizide doğrusal arama (linear search) ve sık yapılan tasarım hataları.
- Eratosthenes kalburu (Sieve of Eratosthenes) ile n'ye kadar olan asal sayıların bulunması ve üç iyileştirme.
- İki sıralı dizinin birleştirilmesi (merging two sorted arrays).
- Bir sonraki derse düşünme sorusu: doğrusal yoklamalı hash tablosunda ortalama yerleşme adımı.

Hocanın Özellikle Vurguladığı Kısımlar

- Algoritmanın çalışması yetmez. Tasarım kurallarına uygun ve verimli olmalıdır.
- Adım sayısı belli olmayan bir iş için `while` türü döngü kullanın. `for` döngüsü kurup `break` ile kırmak yanlış tasarımdır.
- Döngü koşulunda dizi kullanıyorsanız önce dizinin bitip bitmediğini (`i < n`) kontrol edin. Dizi elemanına erişen koşulu sonra yazın.
- Döngüden çıktıktan sonra neden çıktığınızı kontrol edin (bulundu mu, dizi mi bitti).
- İki koşullu durma şartında bağlaç `&&` olmalıdır. `||` kullanmak sık yapılan bir hatadır.
- Kısa olan dizinin önce biteceğini sanmayın. Hangi dizinin önce biteceğini son elemanlar belirler.
- Sahte büyük sayı (gözcü, sentinel) ekleyerek durma şartını kolaylaştırmaya çalışmayın. Gereksiz karmaşa yaratır.
- Önce akış diyagramı veya yarı kod (pseudocode) ile tasarlayın, analiz edin, sonra kodlayın. Kodlandıktan sonra bulunan hatalar "yama" ile kapatılır ve program bozulur.
- Algoritmalar ezberlenmez. Gördüğünüz tasarım fikirlerini başka problemlere uygulamayı öğrenin.
- Problemdeki avantajı kullanın: dizi sıralıysa bundan faydalanın; sırasızsa yapılacak bir şey yoktur, tek tek bakılır.

Kısa Tekrar Notları

- Doğrusal arama koşulu: `while (i < n && a[i] != x) i++;` sonra `if (i == n)` ise eleman yoktur.
- `a[i] != x` önce yazılırsa, eleman yokken `a[n]` okunur. Bu, dizinin dışındaki bir bellek gözüdür.
- Kalbur iyileştirmeleri:
  - 2'nin katlarını baştan ele.

  - Yalnızca $\sqrt{n}$'ye kadar olan sayıların katlarını ele.
  - Asal p için elemeye p^2'den başla.
- Bir sayının katlarını elemeden önce o sayının hâlâ asal işaretli olduğunu (`A[p] != 0`) kontrol et.
- İki sıralı dizinin birleştirilmesinde her eleman bir kez taşındığı için toplam taşıma tam n_a+n_b adımdır; karşılaştırma en fazla n_a+n_b-1'dir. Diziyi arka arkaya ekleyip sıralamak en iyi durumda $O(n \log n)$'dir.
- Algoritmaların verimliliği iki nedenle yeniden önem kazandı: gömülü sistemler ve derin öğrenme (deep learning).

Detaylı Açıklamalar

Dersin amacı ve kapsamı (01:51:18 sonrası)

- Birinci sınıfta ve 2. sınıfın ilk döneminde temel algoritma tasarımı ve C programlama öğrenildi.
- Bu derste de kod gerektiğinde C kullanılır.
- C'nin seçilme nedeni: giriş-çıkış, kontrol yapıları, dosya ve dinamik bellek kullanımı gibi temel yapıları öğretmesi. Temel bir dil iyi bilinirse diğer diller kolay öğrenilir.
- Dersin iki kısmı vardır:
  - **Algoritmalar:** sık kullanılan algoritmalar (sıralama, arama, metin arama) ve bunların tasarım teknikleri.
  - **Veri yapıları:** belirli problem türleri için geliştirilmiş yapılar.
- Algoritmaları öğrenmek düşünme şekli kazandırır. Örneğin quicksort veya bir metin arama algoritmasındaki fikir, değiştirilerek başka bir problemde kullanılabilir.
- Planlanan konular:
  - Temel algoritma tasarımı örnekleri.
  - Algoritma analizine giriş: en iyi, en kötü ve ortalama durum karmaşıklığı. Ayrıntısı 3. sınıftaki Algoritma Analizi dersindedir.
  - Liste (dizi), bağlı listeler (linked lists).
  - Kuyruk ve yığın.
  - Ağaçlar, ikili ağaçlar (binary trees), ikili arama ağaçları (binary search trees).
  - Heap veri yapısı, heap üzerinde sıralama, öncelikli kuyruk (priority queue).
  - Graflar ve minimum kapsayan ağaç (minimum spanning tree).
  - Böl ve yönet (divide and conquer): quicksort, mergesort.
  - Metin (string) arama algoritmaları.
  - Sayarak sıralama (counting sort) gibi diğer sıralama algoritmaları.
- Önerilen kitaplar:
  - Anany Levitin, *Introduction to the Design and Analysis of Algorithms*.
  - Robert Sedgewick (ve Kevin Wayne), *Algorithms*, 4. baskı.
  - Hoca iki kitaptan da derste ve 3. sınıftaki Algoritma Analizi dersinde çok yararlandığını söyler.

Veri yapılarının gerçek hayattaki örnekleri

- **Graf:**
  - Harita uygulamasında yol bulma. Düğümler (nodes) semtlerdir (ör. Kadıköy, Beşiktaş, Davutpaşa). Bağlantılar (edges) mesafe veya süre bilgisi taşır.
  - Sosyal ağlar (ör. Facebook). Düğümler kişiler, bağlantılar kişiler arasındaki ilişkilerdir.
- **Yığın (stack), son giren ilk çıkar (LIFO, last in first out):**
  - Web tarayıcısında "geri" tuşu: en son gelinen sayfaya dönülür.
  - Fonksiyon çağrıları: `main` → `A` → `B` → `C` çağrıldıysa `C` bitince `B`'ye, sonra `A`'ya, sonra `main`'e dönülür.
- **Kuyruk (queue), ilk giren ilk çıkar (FIFO, first in first out):**
  - Ağdaki yazıcıya gönderilen işler sırayla kuyruğa girer ve sırayla yapılır.

Algoritma tasarım adımları

```
1. Problemi anla: ne isteniyor, giriş ne, çıkış ne?
2. Küçük bir örnek çöz:
   - genel durum
   - uç noktalar (ilk eleman, son eleman)
   - özel durum (ör. aranan eleman yok)
3. Algoritmayı tasarla (akış diyagramı veya yarı kod).
4. Doğruluğunu örneklerle kontrol et.
5. Analiz et (zaman ve yer).
6. Sorun varsa 1. veya 3. adıma dön.
7. Ancak analizden memnunsan kodla.
```

- İyi bir algoritmanın ölçütleri:
  - **Verimlilik:** önce zaman, sonra yer.
  - **Basitlik:** kodu okuyan kişi kolayca anlayabilmelidir. Döngüyü `break` ile kırmak okuyanın kafasını karıştırır.
  - **Genellik:** algoritma yalnızca verilen örnek için değil, tüm girişler için çalışmalıdır. Ör. metin arama yalnız küçük harfler için değil, tüm ASCII karakterler için çalışmalıdır.
- Hoca, eskiden donanımın çok kısıtlı olduğunu (256K bellek, disket) anlatır. Sonra donanım gelişince verimlilik bir süre önemsenmedi. Bugün iki nedenle yeniden önemlidir:
  - Gömülü sistemlerde (embedded systems) kaynaklar kısıtlıdır.
  - Derin öğrenmede çok büyük veriyle günlerce, haftalarca çalışan programlar vardır.

Sırasız dizide doğrusal arama (ekrandan, 00:19:30)

- Örnek dizi: `a = {4, 5, 2, 1, 7, 9}`, $N = 6$.
- $x = 7$ (genel durum): 4. indiste bulunur ve durulur.
- $x = 11$ (özel durum): dizinin sonuna kadar gidilir, bulunamaz.
- Yanlış tasarım: `for (i = 0; i < n; i++)` döngüsü içinde `if (a[i] == x) break;`.
  - Algoritma çalışır ama tasarım olarak yanlıştır. Eleman bulununca çıkılacağı baştan bilinir; bu bir sürpriz değildir.
  - Adım sayısı belli olmayan bir iş, adım sayısı belliymiş gibi kurulmuş olur.
- Doğru tasarım:

```
N, a[N] oku
x oku
i = 0
(i < N) && (a[i] != x) olduğu sürece:
    i++
eğer i == N ise:
    "YOK" yazdır
değilse:
    i yazdır
```

- **Koşulların sırası neden önemli?**
  - $x = 11$ için $i = 6$ olduğunda dizi bitmiştir.
  - Önce `a[i] != x` yazılırsa `a[6]` okunur. `int a[6]` tanımında geçerli indisler 0-5'tir. `a[6]` size ait olmayan bir bellek gözüdür.
  - Dizi `int a[50]` olarak tanımlanıp 6 eleman okunduysa, `a[6]` size aittir ama içinde rastgele değer (garbage) vardır.
  - Bu rastgele değer tesadüfen aranan sayıya eşitse algoritma yanlış sonuç verir. Hocanın örneği: uzaya füze gönderen hassas bir sistemde milyarda bir ihtimal bile felakete yol açabilir.
  - Bu yüzden önce `i < n` kontrol edilir. C'de `&&` kısa devre (short-circuit) değerlendirme yapar: ilk koşul yanlışsa ikincisi hiç hesaplanmaz.
- Bu örnekteki üç ders:
  - Adım sayısı belli değilse `while` kullan.
  - Dizi bitebiliyorsa önce bitip bitmediğini kontrol et.
  - Döngüden çıkınca sonucu yazdırmadan önce neden çıktığını kontrol et. Bu bir fonksiyon olsaydı, çağıran taraf da dönen değeri kontrol etmelidir.

Eratosthenes kalburu ile asal sayılar

- Problem: verilen n için 2'den n'ye kadar olan tüm asal sayıları bul. Örnek $n = 25$.
- Asal sayı: yalnızca 1'e ve kendisine bölünebilen sayı. En küçük asal 2'dir.
- **İlk fikir (öğrenci önerisi):** her sayının kendinden önceki sayılara bölünüp bölünmediğine bak.
  - Doğru çalışır. İlk bölen bulununca durulabilir.
  - İyileştirme: yalnızca sayının karekökünе kadar bakmak yeterlidir. Bir sayının karekökten büyük bir çarpanı varsa, karekökten küçük bir eşi de vardır ve o daha önce denenmiştir. Ör. $14 = 2 \cdot 7$; 14 zaten 2'ye bakılırken elenir.
- **Kalbur fikri:** "Bu sayı kime bölünür?" diye sormak yerine "bu sayının katları kim?" diye sor ve katları ele.
  - Bölünebilme kontrolü (`if`) yapılmaz. Katlar `p` adımla ilerlenerek doğrudan silinir.
  - Tek bir dizi yeterlidir: asal kabul edilenler 1, elenenler 0 yapılır.
- $n = 25$ üzerinde adımlar:
  - 2'nin katları elenir: 4, 6, 8, ..., 24. Dizinin yarısından tek seferde kurtulunur.
  - 3 asaldır; elemeye $3^2 = 9$'dan başlanır: 9, 12, 15, 18, 21, 24. (6 zaten 2 tarafından elenmiştir.)
  - 4 elenmiştir; katlarına bakılmaz.
  - 5 asaldır; elemeye $5^2 = 25$'ten başlanır: 25 elenir.
  - $\sqrt{25} = 5$ olduğu için burada durulur. 7'nin katlarına bakmaya gerek yoktur: 14 ve 28'i 2, 21'i 3, 35'i 5 zaten elemiştir; $7^2 = 49 > 25$'tir.
  - Kalan asallar: 2, 3, 5, 7, 11, 13, 17, 19, 23.
- **Üç iyileştirme:**
  1. 2'nin katlarını baştan, kontrol etmeden ele.
  2. Yalnızca $\sqrt{n}$'ye kadar olan sayıların katlarını ele.
  3. Asal p için elemeye $p \cdot p$'den başla; daha küçük katlar zaten daha küçük asallar tarafından elenmiştir.
- Bu iyileştirmeler yapılmasa da algoritma doğru çalışır, ama gereksiz iş yapar.
- Akış diyagramı (ekrandan, 01:08:20):

```
n oku
A[0] = A[1] = A[2] = 1
p = 4'ten n'e kadar 2'şer:      A[p] = 0      (2'nin katlarını ele)
p = 3'ten n'e kadar 2'şer:      A[p] = 1      (tek sayıları asal kabul et)
nn = sqrt(n) + 1
p = 3'ten nn'e kadar 2'şer:
    eğer A[p] != 0 ise:
        j = p * p
        j <= n olduğu sürece:
            A[j] = 0
            j += p
p = 2'den n'e kadar:
    eğer A[p] == 1 ise p yazdır
```

- `nn = sqrt(n) + 1`: hoca karekökün yuvarlanmasında kaçak olmaması için yukarı doğru yuvarlamanın gerekebileceğini söyler.
- `A[0]`, `A[1]`, `A[2]` gözleri içlerinde rastgele değer kalmasın diye 1 yapılır. Yazdırma 2'den başladığı için 0 ve 1 sonuca girmez.

İki sıralı dizinin birleştirilmesi (merging)

- Örnek (ekrandan, 01:22:40): `a = {3, 5, 7, 10, 12, 16}` ($n_a = 6$), `b = {1, 4, 18, 21}` ($n_b = 4$). Sonuç 10 elemanlı `c` dizisidir.
- **İlk akla gelen çözüm:** iki diziyi arka arkaya koy ve herhangi bir sıralama algoritmasıyla sırala.
  - Doğrudur, çalışır, ama verimli değildir.
  - Karşılaştırmalı sıralamanın en iyisi bile $O(n \log n)$'dir. Dizilerin zaten sıralı olması kullanılmamış olur.
- **Sıralı olmanın avantajı:** `a[0] = 3` ile `b[0] = 1` karşılaştırılır. 1 küçükse, a'nın geri kalan elemanlarının hepsinden de küçüktür.

- Yöntem: her dizide bir gösterge (indis) tut. Küçük olanı c'ye yaz ve yalnızca o dizinin göstergesini ilerlet.
- Örnekte adımlar: 1 (b), 3 (a), 4 (b), 5, 7, 10, 12, 16 (a). Burada a biter. b'nin kalanı (18, 21) karşılaştırmadan c'ye taşınır.
- Örnek bilerek seçilmiştir: b kısa olduğu hâlde a önce biter. Dizinin uzunluğu, hangisinin önce biteceğini belirlemez. Son elemanı küçük olan dizi önce biter.
- Akış diyagramı (ekrandan, 01:36:10):

```
na, a[na], nb, b[nb] oku
ia = 0, ib = 0, ic = 0
(ia < na) && (ib < nb) olduğu sürece:
    eğer a[ia] < b[ib] ise:  c[ic++] = a[ia++]
    değilse:                 c[ic++] = b[ib++]
eğer ia < na ise:            (b bitti)
    i = ia'dan na-1'e kadar: c[ic++] = a[i]
değilse:                     (a bitti)
    i = ib'den nb-1'e kadar: c[ic++] = b[i]
c dizisini 0'dan ic-1'e kadar yazdır
```

- `c[ic++] = a[ia++]` ifadesinin açık hâli: önce `c[ic] = a[ia]`, sonra `ia++` ve `ic++`.
- Durma şartında `&&` kullanılmalıdır. "Dizilerden biri bitince çık" denmek istenir. `||` yazılırsa dizilerden biri bittiği hâlde döngü devam eder.
- Eşit elemanlar varsa: koşul `a[ia] < b[ib]` olduğu için eşitlikte önce b'deki eleman taşınır, sonraki adımda a'daki eşit eleman taşınır. İkisi de sonuçta yer alır.
- Gereksiz yaklaşımlar:
  - Önce son elemanları karşılaştırıp hangi dizinin önce biteceğine göre ayrı döngüler kurmak.
  - Dizinin sonuna sahte, çok büyük bir sayı (ör. en büyük `int`) eklemek.
- Karmaşıklık: hoca döngünün en kötü ihtimalle $n_a + n_b$ kez döneceğini söyler (01:38:34). Kalan elemanların taşınması da sayılınca taşıma her durumda tam $n_a + n_b$'dir; karşılaştırma sayısı en fazla $n_a + n_b - 1$'dir. Bu, birleştirip sıralamaktan daha verimlidir.
- Bir öğrencinin önerdiği fikir de aynıdır: bir dizi bitince diğerinin kalanını karşılaştırmadan kopyala.

C karşılıkları

Derste C kodu yazılmadı. Aşağıdaki fonksiyonlar ekrandaki akış diyagramlarından ajan tarafından çevrilmiştir. `gcc -Wall` (`-lm` ile) derlenip derste kullanılan örneklerle çalıştırılmıştır. Çıktılar: `7 -> 4, 11 -> -1`; `2 3 5 7 11 13 17 19 23`; `1 3 4 5 7 10 12 16 18 21`.

```c
#include <stdio.h>
#include <math.h>

/* Sırasız dizide doğrusal arama: bulursa indisi, bulamazsa -1 döndürür. */
int dogrusal_ara(int a[], int n, int x) {
    int i = 0;
    while (i < n && a[i] != x)   /* önce dizinin bitip bitmediği kontrol edilir */
        i++;
    if (i == n)
        return -1;
    return i;
}

/* Eratosthenes kalburu: 2..n arasındaki asalları yazdırır. a dizisi en az n+1 gözlü olmalı. *
void kalbur(int a[], int n) {
    int p, j, nn;
    if (n < 2) {                    /* n < 2 ise a[2] dizi dışında kalabilir ve asal yoktur *
        printf("\n");
        return;
    }
    a[0] = a[1] = a[2] = 1;
```

```c
    for (p = 4; p <= n; p += 2)      /* 2'nin katlarını ele */
        a[p] = 0;
    for (p = 3; p <= n; p += 2)      /* tek sayıları başlangıçta asal say */
        a[p] = 1;
    nn = (int)sqrt(n) + 1;
    for (p = 3; p <= nn; p += 2) {
        if (a[p] != 0) {             /* yalnız asal sayının katlarını ele */
            for (j = p * p; j <= n; j += p)
                a[j] = 0;
        }
    }
    for (p = 2; p <= n; p++)
        if (a[p] == 1)
            printf("%d ", p);
    printf("\n");
}

/* Sıralı a ve b dizilerini sıralı c dizisinde birleştirir; c'nin eleman sayısını döndürür. */
int birlestir(int a[], int na, int b[], int nb, int c[]) {
    int ia = 0, ib = 0, ic = 0, i;
    while (ia < na && ib < nb) {
        if (a[ia] < b[ib])
            c[ic++] = a[ia++];
        else
            c[ic++] = b[ib++];
    }
    if (ia < na)                     /* b bitti: a'nın kalanını taşı */
        for (i = ia; i < na; i++)
            c[ic++] = a[i];
    else                             /* a bitti: b'nin kalanını taşı */
        for (i = ib; i < nb; i++)
            c[ic++] = b[i];
    return ic;
}

int main(void) {
    int d[] = {4, 5, 2, 1, 7, 9};
    int asal[26];
    int a[] = {3, 5, 7, 10, 12, 16}, b[] = {1, 4, 18, 21}, c[10];
    int k, nc;

    printf("7 -> %d, 11 -> %d\n", dogrusal_ara(d, 6, 7), dogrusal_ara(d, 6, 11));
    kalbur(asal, 25);
    nc = birlestir(a, 6, b, 4, c);
    for (k = 0; k < nc; k++)
        printf("%d ", c[k]);
    printf("\n");
    return 0;
}
```

Düşünme sorusu: hash tablosunda ortalama yerleşme adımı (ekrandan, 01:47:00)

- Kurulum:
  - $N = 10$ gözlü, başlangıçta sıfırlanmış bir tablo vardır.
  - Bir x sayısı `x % 10` adresine (ev adresi) yerleşmeye çalışır.
  - Göz doluysa bir sonraki göze bakılır. Sondan sonra başa dönülür. Bu, hashing yöntemlerinden biridir.
- Örnek: 2, 3, 22, 9, 19 sırasıyla gelir.
  - 2 → 2. göz (1. deneme). 3 → 3. göz (1. deneme).
  - 22 → 2 ve 3 dolu, 4. göze (3. deneme).

  - 9 → 9. göz (1. deneme). 19 → 9 dolu, başa dönüp 0. göze (2. deneme).
- Soru: tablodaki elemanlar ortalama kaçıncı denemede yerleşmiştir?
- Eleman sayısı tablo boyutundan çok küçüktür (yaklaşık yarısı). Tablo dolmaz.
- Soru Ders 2'de çözülür.

Ders sonrası notlar

- Hoca gözcü (sentinel) örneğinde en büyük tam sayı olarak 32.767 değerini söyler. Bu, 16 bitlik `int` için geçerlidir. Bugünkü derleyicilerde `int` genelde 32 bittir ve en büyük değer 2.147.483.647'dir. Taşınabilir kodda `<limits.h>` içindeki `INT_MAX` kullanılır.
- "Sıralama algoritmalarının en iyisi $O(n \log n)$'dir" ifadesi karşılaştırmalı sıralama (comparison sort) için geçerlidir. Sayarak sıralama gibi algoritmalar, belirli koşullarda doğrusal zamanda çalışabilir. Bu algoritmalar dersin ilerleyen haftalarında işlenir.
- Hoca Eratosthenes için "M.Ö. 200 falan olabilir" diye tahminde bulunur (00:32:48). Bu tarih altyazıdaki kaynaklarla doğrulanamadı; kesin bilgi olarak alınmamalıdır.
- Kalburda dizi 0'dan n'ye kadar indislendiği için `A` dizisi en az $n + 1$ gözlü olmalıdır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.